Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 6. Seguridad en el desarrollo de las aplicaciones web

Índice

Esquema

Ideas clave

6.1. Introducción y objetivos

6.2. Principios del desarrollo seguro

6.3. Desarrollo seguro

6.4. Referencias bibliográficas

A fondo

Vulnerabilidades en el código fuente: sitio web de Fortify

Seguridad en frameworks

Vulnerabilidades en el código fuente: sitio web de Find Security Bugs

OWASP ESAPI project

OWASP Java Encoder Project

Test

Esquema

© Universidad Internacional de La Rioja (UNIR)

6.1. Introducción y objetivos

El desarrollo de aplicaciones se ha convertido en una pieza central de la sociedad moderna. Desde servicios bancarios hasta redes sociales, dependemos de *software* para casi todas nuestras actividades. Sin embargo, esta dependencia trae consigo un riesgo inherente: las vulnerabilidades de seguridad. Una aplicación mal diseñada puede ser una puerta de entrada para ataques cibernéticos que ponen en riesgo la información personal, financiera y la integridad de los sistemas.

En este contexto, el desarrollo seguro de aplicaciones no es solo una práctica recomendada, sino una responsabilidad ética y profesional. A menudo, la seguridad se considera una característica que se añade al final del ciclo de desarrollo, una mentalidad que ha demostrado ser costosa y poco efectiva. Las vulnerabilidades descubiertas en producción requieren parches de emergencia que generan gastos inesperados y dañan la reputación de las empresas. La seguridad debe ser un pilar fundamental desde la fase de diseño, pasando por la implementación, las pruebas y el despliegue.

Este enfoque proactivo se conoce como Security by Design, una metodología que integra la ciberseguridad en cada etapa del ciclo de vida del desarrollo de *software* (SDLC). No se trata solo de escribir código libre de errores, sino de comprender las amenazas, aplicar los principios de seguridad adecuados y utilizar herramientas y procesos que fortalezcan la resiliencia de la aplicación frente a ataques.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Para los profesionales de la ciberseguridad, dominar el desarrollo seguro es crucial. No basta con saber cómo penetrar un sistema (*hacking* ético) o cómo responder a un incidente, sino que es fundamental comprender cómo se construyen las aplicaciones para poder identificar, prevenir y mitigar las vulnerabilidades desde su origen.

Este tema está diseñado para explorar en profundidad las metodologías, principios y herramientas esenciales para construir *software* robusto y seguro. Se abordarán temas clave, como la gestión de dependencias, el cifrado de datos, la validación de entradas de usuario y la gestión de secretos, entre otros. Se aprenderá a pensar como un atacante para proteger las aplicaciones de las amenazas más comunes, como la inyección SQL, el *cross-site scripting* (XSS) y los fallos de configuración. El objetivo es que, al finalizar, sean capaces de liderar la integración de la seguridad en cualquier equipo de desarrollo, transformándose en arquitectos de *software* que no solo construyen funcionalidades, sino también confianza y resiliencia en el mundo digital.

Los objetivos que se pretenden conseguir son:

- Conocer los principales principios y buenas prácticas de seguridad que tener en cuenta al desarrollar código fuente.
- Comprender las vulnerabilidades en el código fuente de las aplicaciones web mediante ejemplos concretos de cada vulnerabilidad.
- Aprender a desarrollar código fuente de forma segura sin cometer vulnerabilidades de seguridad CWE Top 25, validando los campos de entrada a la aplicación web y codificando las salidas.

© Universidad Internacional de La Rioja (UNIR)

6.2. Principios del desarrollo seguro

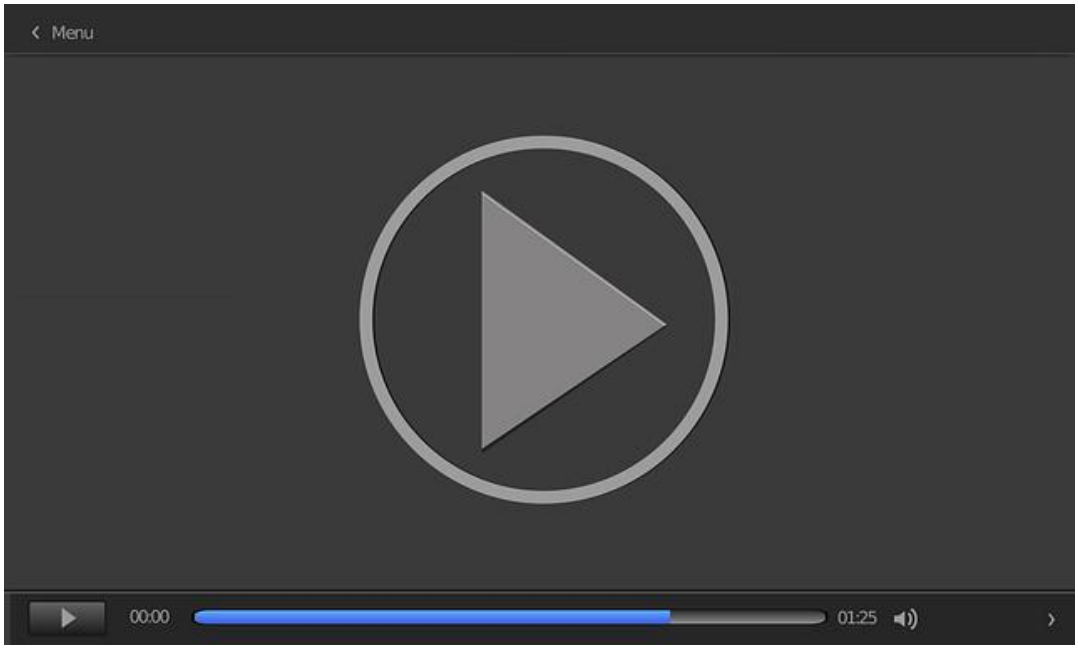
La práctica principal que se debe realizar durante la **fase de implementación,** en cuanto a consecución de código seguro se refiere, es la de **revisión estática de código.** Esta práctica o actividad se considera la de más importancia de las siete del modelo de ciclo de vida de desarrollo de *software* seguro (*systems development life cycle*, SDLC), según lo que Gary McGraw (2006) explica en su libro *Software security: building security in*. En este apartado se van a explicar, en primer lugar, los defectos más comunes que se pueden cometer en aplicaciones web que se desarrollan siguiendo las especificaciones Java 2 Enterprise Edition (J2EE).

Encontrar *bugs* tempranamente conlleva una reducción del coste económico en varios órdenes frente a si se descubrieran en fases posteriores. Este hecho tiene que incitar a las organizaciones a poner especial interés en esta actividad y encuadrarla en los procesos y procedimientos de desarrollo.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Se recomienda visualizar el vídeo *Principios del desarrollo seguro de* software sobre buenas prácticas para tener en cuenta en el desarrollo seguro de *software* y OWASP *cheat sheet input validation.*



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=737ae24d-bf44-4501-8d38-b0c700cc78e2

Recomendaciones de codificación segura

Es preciso comenzar el proceso de formación en implementación de código de forma segura dando una serie de **recomendaciones generales** para tener en cuenta como punto de partida o como línea que seguir en la profundización del conocimiento de la seguridad del código. Una buena serie de recomendaciones se da en los libros *Secure coding: principles and practices* (Graff y Van Wyk, 2003), (Chess, 2007) y OWASP Secure Coding Practices (OWASP sec, s.f.). Estas se dividen en varias categorías.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Según Benjamín Franklin, «desconfianza y precaución son los padres de la seguridad».

Validación de entrada

- Realizar todas las **validaciones de entrada** en un sistema de confianza (del lado del servidor, no del lado del cliente).
- Identificar todas las **fuentes** de datos y **clasificarlas** en confiables y no confiables.
- Validar todos los datos de **fuentes no confiables** (bases de datos, flujos de archivos, etc.).
- Utilizar una **rutina de validación** de entrada centralizada para toda la aplicación.
- Especificar **conjuntos de caracteres,** como UTF-8, para todas las fuentes de entrada (canonicalización).
- Codificar las entradas en un conjunto de **caracteres común** antes de validarlas.
- Todos los fallos de validación deben dar lugar al **rechazo de la entrada.**
- Si el sistema admite conjuntos de **caracteres extendidos** UTF-8, validar después de completar la decodificación UTF-8.
- **Validar** todos los datos proporcionados por el cliente **antes de procesarlos.**
- Verificar que los valores de los encabezados de protocolo, tanto en las solicitudes como en las respuestas, contengan solo **caracteres ASCII.**
- Valide los datos de las **redirecciones.**
- Validar los tipos de **datos esperados** utilizando una lista de «permitidos» en lugar de una lista de «denegados».
- Validar el **rango** de datos.

© Universidad Internacional de La Rioja (UNIR)

- Validar la **longitud** de los datos.
- Si se debe permitir alguna entrada potencialmente peligrosa, implementar **controles adicionales.**
- Si la rutina de validación estándar no puede abordar algunas entradas, utilizar **comprobaciones discretas adicionales.**
- Utilizar la canonización para abordar los **ataques de ofuscación.** Los atacantes a menudo utilizan la ofuscación para disfrazar su código empleando diversas técnicas, como la codificación hexadecimal, UTF-8 o URL-encoding, para que el sistema de detección de intrusiones no lo reconozca como una amenaza. La canonización actúa como un desenmascarador, ya que revierte estas codificaciones a su **formato original** y hace visible el verdadero contenido. Por ejemplo, un atacante podría intentar inyectar el comando ../etc/passwd (que busca acceder a un archivo de contraseñas) utilizando una codificación URL, como %2E%2E%2Fetc%2Fpasswd. Cuando el sistema aplica la canonización, **decodifica** esta cadena, la convierte a ../etc/passwd y la **compara** con una lista de patrones de ataque conocidos. Si esta cadena coincide con un patrón de ataque, el sistema puede bloquear la solicitud y proteger el servidor.

Codificación de salida

- Realizar toda la **codificación de salida** en un sistema de confianza (del lado del servidor, no del lado del cliente).
- Utilizar una **rutina estándar** y **probada** para cada tipo de codificación de salida.
- Especificar **conjuntos de caracteres,** como UTF-8, para todas las salidas.
- **Codificar contextualmente** todos los datos devueltos al cliente desde fuentes no fiables.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Asegurarse de que la codificación de salida sea **segura** para **todos los sistemas** de destino.
- **Limpiar** contextualmente toda la **salida de datos** no fiables a consultas SQL, XML y LDAP.
- Limpiar toda la salida de datos no fiables a **comandos** del sistema operativo.

Prácticas criptográficas

- Todas las funciones criptográficas utilizadas para proteger los secretos del usuario de la aplicación deben implementarse en un **sistema de confianza.**
- Proteger los secretos del **acceso no autorizado.**
- Los módulos criptográficos deben **fallar** de **forma segura.**
- Todos los números aleatorios, nombres de archivos aleatorios, GUID aleatorios y cadenas aleatorias deben generarse utilizando el **generador de números aleatorios** aprobado del módulo criptográfico.
- Los módulos criptográficos utilizados por la aplicación deben cumplir con la **norma FIPS 140-2** o una norma equivalente.
- Establecer y utilizar una política y un proceso para la gestión de las **claves criptográficas.**

© Universidad Internacional de La Rioja (UNIR)

Gestión de errores y registro

- **No revelar información confidencial** en las respuestas de error, incluidos los detalles del sistema, los identificadores de sesión o la información de la cuenta.
- **Utilizar gestores de errores** que no muestren información de depuración o de seguimiento de la pila.
- **Implementar mensajes de error** genéricos y utilizar páginas de error personalizadas.
- **La aplicación debe gestionar los errores** de la aplicación y no depender de la configuración del servidor.
- **Liberar correctamente la memoria** asignada cuando se produzcan condiciones de error.
- La lógica de gestión de errores asociada a los controles de seguridad debe **denegar el acceso de forma predeterminada.**
- Todos los **controles de registro** deben implementarse en un sistema de confianza.
- Los controles de registro deben admitir tanto el **éxito** como el **fracaso** de los eventos de seguridad especificados.
- Asegurarse de que los registros contengan **datos importantes** sobre los eventos registrados.
- Asegurarse de que las entradas de registro que incluyan **datos no fiables no se ejecuten** como código en la interfaz o el *software* de visualización de registros previstos.
- **Restringir el acceso** a los registros solo a las personas autorizadas.
- Utilizar una **rutina central** para todas las operaciones de registro.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **No almacenar información confidencial en los registros,** incluidos detalles innecesarios del sistema, identificadores de sesión o contraseñas.
- Asegurarse de que exista un **mecanismo** para realizar **análisis de registros.**
- Registrar todos los **errores** de **validación de entradas.**
- Registrar todos los **intentos de autenticación,** especialmente los fallos.
- Registrar todos los **fallos de control de acceso.**
- Registrar todos los **eventos de manipulación** aparentes, incluidos los cambios inesperados en los datos de estado.
- Registrar los **intentos de conexión** con *tokens* de sesión no válidos o caducados.
- Registre todas las **excepciones** del sistema.
- Registrar todas las **funciones administrativas,** incluidos los cambios en la configuración de seguridad.
- Registrar todos los **fallos de conexión TLS** del *backend.*
- Registrar los fallos del **módulo criptográfico.**
- Utilizar una función *hash* criptográfica para validar la **integridad de las entradas** del registro.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

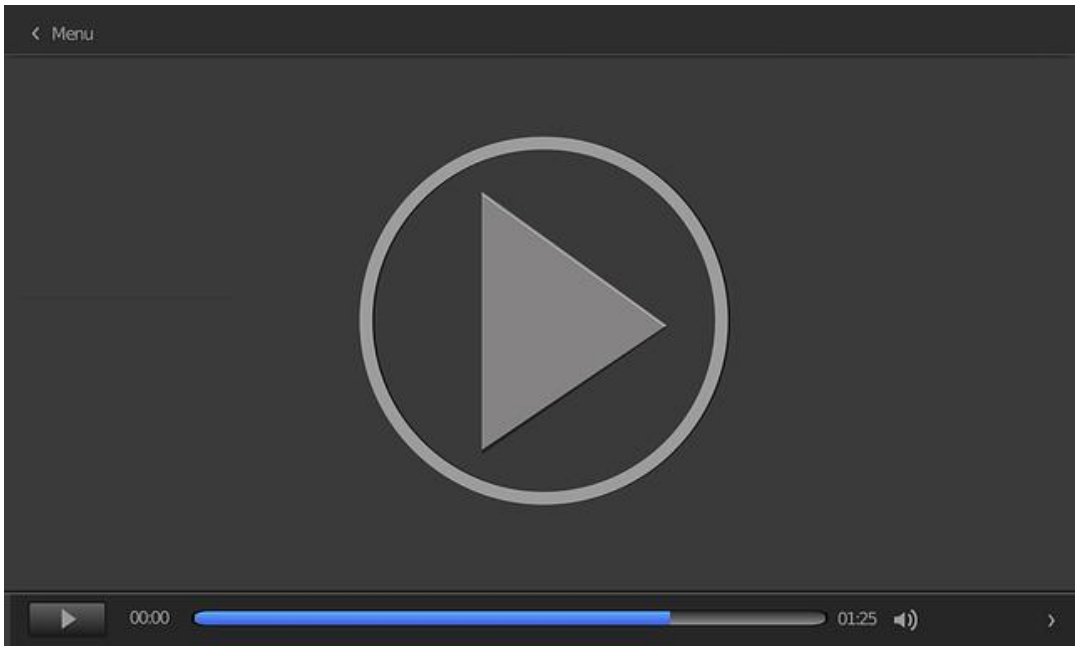
Gestión de archivos

- **No pasar los datos** proporcionados por el usuario directamente a ninguna función de **inclusión dinámica.**
- **Exigir la autenticación** antes de permitir la carga de un archivo.
- **Limitar el tipo de archivos** que se pueden cargar solo a aquellos que sean necesarios para fines comerciales.
- Validar que los archivos cargados sean del tipo esperado y **comprobar los encabezados** de los archivos en lugar de la extensión.
- No guardar archivos en el mismo **contexto web** que la aplicación.
- Impedir o **restringir** la **carga de cualquier archivo** que pueda ser interpretado por el servidor web.
- **Desactivar los privilegios de ejecución** en los directorios de carga de archivos.
- Cuando haga referencia a archivos existentes, utilizar una lista de **nombres** y **tipos de archivos permitidos.**
- **No pasar datos proporcionados por el usuario** a una redirección dinámica.
- **No pasar rutas de directorios o archivos.** Utilice valores de índice asignados a una lista predefinida de rutas.
- **Nunca enviar la ruta absoluta** del archivo al cliente.
- Asegurarse de que los archivos y recursos de la aplicación sean de **solo lectura.**
- Analizar los archivos subidos por los usuarios en busca de **virus** y *malware.*

© Universidad Internacional de La Rioja (UNIR)

6.3. Desarrollo seguro

Se recomienda visualizar el vídeo *Find Security Bugs* para ver una explicación de la herramienta de análisis estático de seguridad del código fuente para lenguaje Java Find Security Bugs.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=027adb36-ed28-408f-9d52-b0c700cbe6a7

Se pueden consultar ejemplos adicionales de código Java con vulnerabilidades de seguridad en Bermejo (2021), Static code analysis tools for your Java (s. f.), Predictable pseudorandom number generator (s. f.), Debilidades (s.f.), Hoffman (2024) y Macdonald (2024). A continuación, se muestran ejemplos de **vulnerabilidades** en lenguaje Java (Servlet) y su solución.

© Universidad Internacional de La Rioja (UNIR)

SQL injection (SQLi) CWE 89

Es fundamental evitar las vulnerabilidades de inyección SQL (SQLi) en Java Servlets. La solución principal, y más efectiva, es utilizar **sentencias preparadas** *(PreparedStatement)* en lugar de concatenar cadenas para construir consultas SQL.

Una aplicación Java Servlet es vulnerable a SQLi cuando construye una consulta SQL directamente **concatenando la entrada del usuario** (que no ha sido validada o sanitizada). En este ejemplo, la entrada del usuario (user_id) se toma de un parámetro de la **solicitud HTTP** y se concatena directamente en la cadena SQL:

```java
import java.sql.Connection;
import java.sql.Statement;
import java.sql.ResultSet;

// ... dentro del método doGet o doPost de tu Servlet

String userId = request.getParameter("user_id"); // Entrada del usuario
String sql = "SELECT * FROM users WHERE id = '" + userId + "'";

try (Connection conn = getConnection();
     Statement stmt = conn.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    // Procesar resultados...

} catch (Exception e) {
    // Manejo de errores
}
```

Figura 1. SQLI. Fuente: elaboración propia.

Explotación: si un atacante introduce la siguiente cadena maliciosa como valor para el parámetro user_id: ' OR '1'='1, la **consulta SQL final** que se ejecutaría en la base de datos sería: SELECT * FROM users WHERE id = '' OR '1'='1' Dado que la condición '1'='1' siempre es verdadera, la cláusula WHERE se evalúa como verdadera para todas las filas, lo que permite al atacante recuperar la información de todos los usuarios en lugar de solo la de un usuario específico.

© Universidad Internacional de La Rioja (UNIR)

Solución: la solución más robusta y estándar en Java para prevenir SQLi es utilizar la clase *PreparedStatement*. Esta clase precompila la consulta SQL en la base de datos, lo que significa que el código SQL se define antes de que se introduzcan los datos del usuario. Los datos del usuario se pasan a la consulta como parámetros (utilizando el carácter ?) y son tratados por la base de datos como **valores de datos,** nunca como parte del comando SQL ejecutable.

```java
import java.sql.Connection;
import java.sql.PreparedStatement; // ¡Clave!
import java.sql.ResultSet;

// ... dentro del método doGet o doPost de tu Servlet

String userId = request.getParameter("user_id"); // Entrada del usuario

// Usamos '?' como marcador de posición para la entrada
String sql = "SELECT * FROM users WHERE id = ?";

try (Connection conn = getConnection();
     PreparedStatement pstmt = conn.prepareStatement(sql)) { // 1. Prepara la sentencia

    // 2. Asigna la entrada del usuario como un parámetro (valor de dato)
    pstmt.setString(1, userId); // El '1' es el índice del primer '?'

    // 3. Ejecuta la sentencia ya segura
    try (ResultSet rs = pstmt.executeQuery()) {
        // Procesar resultados...
    }

} catch (Exception e) {
    // Manejo de errores
}
```

Figura 2. Solución SQLI. Fuente: elaboración propia.

Cross Site Scripting (XSS) CWE 79

Un ejemplo común de vulnerabilidad de *cross-site scripting* (XSS) reflejado en Java se produce al tomar directamente un parámetro de una **solicitud HTTP** y mostrándolo en la página web sin realizar una **codificación de salida** adecuada.

Ejemplo de vulnerabilidad XSS: imaginar un ErrorServlet que muestra un mensaje de error o una notificación usando un parámetro de URL llamado mensaje:

© Universidad Internacional de La Rioja (UNIR)

```java
// Archivo: ErrorServlet.java

import java.io.*;
import javax.servlet.http.*;

public class ErrorServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // 🚨 VULNERABILIDAD AQUÍ: Se toma la entrada del usuario
        // y se escribe directamente en el HTML.
        String mensaje = request.getParameter("mensaje");

        out.println("<html>");
        out.println("<head><title>Error</title></head>");
        out.println("<body>");
        out.println("<h1>Notificación</h1>");
        // La inyección se produce en esta línea:
        out.println("<div style='color: red;'>" + mensaje + "</div>");
        out.println("</body></html>");
    }
}
```

Figura 3. XSS. Fuente: elaboración propia.

Explotación: un atacante crea un enlace malicioso inyectando un *payload* de JavaScript en el parámetro mensaje:

http://ejemplo.com/ErrorServlet?mensaje=
<script>alert('XSS_Robo_de_Cookies');</script>

Resultado del ataque: cuando una víctima hace clic en el enlace, el servidor devuelve el siguiente HTML al navegador:

```html
<h1>Notificación</h1>
<div style='color: red;'><script>alert('XSS_Robo_de_Cookies');</script></div>
```

Figura 4. OWASP Java Encoder dependency. Fuente: elaboración propia.

El navegador ejecuta el *script*, que podría usarse para robar la *cookie* de sesión del usuario, registrar pulsaciones de teclas o redirigirlo a un sitio de *phishing*.

© Universidad Internacional de La Rioja (UNIR)

Solución: codificación de salida *(output encoding):* la defensa fundamental contra el XSS es la codificación de salida *(output encoding).* Esto convierte los caracteres especiales HTML (<, >, ", ', &) en sus entidades HTML (<, >, etc.) para que el navegador los muestre como **texto simple,** en lugar de interpretarlos como código ejecutable.

Solución segura con OWASP Java Encoder: se recomienda usar una librería de seguridad probada como **OWASP Java Encoder** para garantizar una codificación correcta en el contexto apropiado.

- **Añadir la dependencia:** (usando Maven)

```
<dependency>
    <groupId>org.owasp.encoder</groupId>
    <artifactId>encoder</artifactId>
    <version>1.2.3</version>
    </dependency>
```

Figura 5. OWASP Java Encoder dependency. Fuente: elaboración propia.

- **Modificar el Servlet para codificar:** utiliza el método Encode.forHtml() para codificar el mensaje antes de escribirlo en el HTML.

© Universidad Internacional de La Rioja (UNIR)

```java
// Archivo: ErrorServlet.java (SEGURO)

import java.io.*;
import javax.servlet.http.*;
// Importar la librería de codificación
import org.owasp.encoder.Encode;

public class ErrorServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String mensajeInseguro = request.getParameter("mensaje");

        // ✅ SOLUCIÓN: Codificar la entrada para el contexto HTML antes de mostrarla
        String mensajeSeguro = Encode.forHtml(mensajeInseguro);

        out.println("<html>");
        out.println("<head><title>Error</title></head>");
        out.println("<body>");
        out.println("<h1>Notificación</h1>");
        // Se imprime la versión codificada:
        out.println("<div style='color: red;'>" + mensajeSeguro + "</div>");
        out.println("</body></html>");
    }
}
```

Figura 6. Solución codificación de la salida con OWASP Java Encoder. Fuente: elaboración propia.

Resultado de la solución:

Con la codificación, el ataque ahora genera el siguiente HTML:

```
<h1>Notificación</h1>
<div style='color:
ed;'>&lt;script&gt;alert(&#x27;XSS_Robo_de_Cookies&#x27;);&lt;/script&gt;</div>
```

Figura 7. Solución codificación de la salida con OWASP Java Encoder. Fuente: elaboración propia.

El navegador interpreta < como el carácter <, y > como >, por lo que muestra el *payload* como **texto inofensivo** en lugar de ejecutarlo como un *script*.

© Universidad Internacional de La Rioja (UNIR)

Estrategia de defensa adicional: aunque la codificación de salida es la defensa clave, se recomienda usar también:

- **Validación estricta de entrada:** si el dato de entrada solo debe ser un número o un texto simple, valida y rechaza cualquier carácter especial antes de procesar el dato.
- ***Content security policy* (CSP):** implementa un encabezado HTTP de CSP para decirle al navegador qué fuentes de *scripts* y estilos son seguras. Esto proporciona una capa de protección adicional si la codificación falla.

Resumen de la solución con OWASP Java encoder

Tipo de XSS	Ubicación de inyección	Solución de codificación (OWASP *encoder*)
En el cuerpo del HTML (como el ejemplo)	Dentro de etiquetas , <h1>, etc.	Encode.forHtml(entrada)
Dentro de una etiqueta HTML (ej. atributo *value*)	<input value="...">	Encode.forHtmlAttribute(entrada)
Dentro de JavaScript (ej. en una función)	<script>var a = '...';</script>	Encode.forJavaScript(entrada)

Tabla 1. OWASP Java *encoder*. Fuente: elaboración propia.

Cross-site request forgery (CSRF) CWE352

La vulnerabilidad de *cross-site request forgery* **(CSRF)** en un Java Servlet ocurre cuando el Servlet realiza una **acción sensible** basándose únicamente en la información de la sesión del usuario (como las *cookies*), sin verificar que la solicitud provenga de la **aplicación legítima.** La solución más robusta y común es implementar Tokens anti-CSRF únicos por sesión.

Ejemplo de vulnerabilidad CSRF: imagina un Servlet que procesa la eliminación de una cuenta (una acción de cambio de estado) mediante una solicitud HTTP **POST**. Este Servlet asume que cualquier solicitud POST con la *cookie* de sesión válida es legítima.

© Universidad Internacional de La Rioja (UNIR)

```java
// Archivo: EliminarCuentaServlet.java

import java.io.IOException;
import javax.servlet.http.*;

public class EliminarCuentaServlet extends HttpServlet {

    // Este método es vulnerable si no hay protección CSRF
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        // Simulación: La lógica de la aplicación obtiene el ID de usuario de la sesión
        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("usuarioId") != null) {
            String userId = (String) session.getAttribute("usuarioId");

            // 🚨 VULNERABILIDAD AQUÍ: La solicitud se procesa
            // sin verificar el origen (token Anti-CSRF).

            // Lógica para eliminar la cuenta del usuario
            System.out.println("Eliminando cuenta para el Usuario ID: " + userId);

            // Lógica de respuesta...
            response.sendRedirect("confirmacion_eliminacion.jsp");
        } else {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "No autenticado");
        }
    }
}
```

Figura 8. CSRF. Fuente: elaboración propia.

Un atacante podría crear una **página maliciosa** con una etiqueta <img> que, al ser visitada por un usuario del banco, envía automáticamente una solicitud para transferir dinero sin su conocimiento.

Explotación:

- **Víctima autenticada:** un usuario (Alice) inicia sesión en el sitio web legítimo (https://banco.com). El navegador guarda la *cookie* de sesión.
- **Página maliciosa:** el atacante (Bob) crea una página web maliciosa (https://sitio-malicioso.com) que contiene un formulario invisible o código JavaScript que realiza una solicitud POST automática a la URL vulnerable.

© Universidad Internacional de La Rioja (UNIR)

```html
<html>
    <body onload="document.forms[0].submit()">
        <form action="https://banco.com/EliminarCuentaServlet" method="POST">
            </form>
        <p>¡Has ganado un premio! (El formulario se envía automáticamente)</p>
    </body>
</html>
```

Figura 9. Petición maliciosa servidor atacante. Fuente: elaboración propia.

- **Ejecución:** Bob engaña a Alice para que visite https://sitio-malicioso.com mientras su sesión en https://banco.com sigue activa.
- **Resultado:** el navegador de Alice envía automáticamente la solicitud POST al EliminarCuentaServlet e incluye automáticamente la *cookie* de sesión válida de https://banco.com. El Servlet ve una solicitud válida con una sesión activa y procede a eliminar la cuenta de Alice sin su consentimiento.
- **Solución:** *tokens* anti-CSRF sincronizados. La solución estándar es usar un *token* anti-CSRF (o sincronizado): un valor secreto, aleatorio y único asociado a la sesión del usuario que se incluye en las solicitudes de cambio de estado.

1. Generar y almacenar el *token* (doGet): en el Servlet o la vista que renderiza el formulario, se genera el *token* y se guarda en la sesión.

© Universidad Internacional de La Rioja (UNIR)

```java
// Archivo: GenerarTokenServlet (o parte del Servlet que sirve la vista)

import java.security.SecureRandom;
import java.math.BigInteger;
// ... otros imports

public class GenerarTokenServlet extends HttpServlet {
    private SecureRandom random = new SecureRandom();

    protected void doGet(HttpServletRequest request, HttpServletResponse response) {

        HttpSession session = request.getSession();

        // 1. Generar un token criptográficamente seguro
        String csrfToken = new BigInteger(130, random).toString(32);

        // 2. Almacenar el token en la sesión del usuario
        session.setAttribute("CSRF_TOKEN", csrfToken);

        // 3. Pasar el token a la vista (JSP)
        request.setAttribute("csrfToken", csrfToken);

        // ... Redirigir a la JSP que contiene el formulario
        // request.getRequestDispatcher("eliminar_formulario.jsp").forward(request,
response);
    }
}
```

Figura 10. Generar token anti CSRF. Fuente: elaboración propia.

2. Incluir el *token* en el formulario (JSP): la vista incluye el *token* como un campo oculto en el formulario.

```jsp
<form action="EliminarCuentaServlet" method="POST">
    <input type="hidden" name="csrfToken" value="${csrfToken}" />
    <button type="submit">Confirmar Eliminación de Cuenta</button>
</form>
```

Figura 11. Incluir token anti CSRF en formulario. Fuente: elaboración propia.

3. Validar el *token* (doPost Seguro): el Servlet que procesa la acción debe verificar que el *token* recibido coincida con el almacenado en la sesión.

© Universidad Internacional de La Rioja (UNIR)

```java
// Archivo: EliminarCuentaServlet.java (SEGURO)

public class EliminarCuentaServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("usuarioId") == null) {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "No autenticado");
            return;
        }

        // 1. Obtener el token de la solicitud (formulario)
        String requestToken = request.getParameter("csrfToken");

        // 2. Obtener el token almacenado en la sesión
        String sessionToken = (String) session.getAttribute("CSRF_TOKEN");

        // 🔑 SOLUCIÓN: Comparar ambos tokens.
        if (requestToken == null || !requestToken.equals(sessionToken)) {
            // El token no coincide o falta: ¡Posible ataque CSRF!
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Solicitud no válida
(CSRF detectado)");
            return;
        }

        // Si los tokens coinciden, la solicitud es legítima.
        String userId = (String) session.getAttribute("usuarioId");
        System.out.println("Eliminando cuenta para el Usuario ID: " + userId);

        // ... Lógica de respuesta ...
        response.sendRedirect("confirmacion_eliminacion.jsp");
    }
}
```

Figura 12. Validar *token* anti CSRF en formulario. Fuente: elaboración propia.

Dado que el atacante (Bob) **no puede leer el *token* aleatorio** de la sesión de Alice debido a la política de mismo origen *(same-origin policy)*, no puede incluir el valor correcto en su formulario malicioso y la solicitud es rechazada.

Alternativas de solución:

- **SameSite *cookies:*** configurar las *cookies* de sesión con el atributo SameSite=Strict o Lax (según el caso de uso) es una defensa eficaz y moderna, ya que evita que el navegador envíe *cookies* de sesión a sitios de origen cruzado en solicitudes POST.
- **OWASP CSRFGuard:** para aplicaciones más grandes, utiliza una librería de *framework,* como OWASP CSRFGuard, que automatiza la generación y validación del *token,* a menudo implementado como un Servlet Filter.

© Universidad Internacional de La Rioja (UNIR)

- **Cabecera content security policy.**

Path traversal CWE 22

La vulnerabilidad *path traversal* (o *directory traversal*) en un Java Servlet surge al usar una **entrada de usuario no validada** para construir una ruta de archivo, lo que permite a un atacante acceder a archivos fuera del directorio de la aplicación, típicamente con secuencias como ../. A continuación, se muestra un ejemplo de código vulnerable y una solución segura que combina la **lista blanca** y la **normalización de ruta** (usando getCanonicalPath() o Path.toRealPath()). Imagina un servicio web que permite a los usuarios descargar archivos, como imágenes de perfil, basándose en el nombre de archivo que proporcionan.

```java
import java.io.*;
import javax.servlet.http.*;

public class VulnerableServlet extends HttpServlet {
    private static final String BASE_DIR = "/var/www/data/docs/"; // Directorio de archivos

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        // 1. Obtener la entrada del usuario (NO VALIDADA)
        String filename = request.getParameter("filename");

        if (filename == null || filename.isEmpty()) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST);
            return;
        }

        // 2. Construcción directa de la ruta del archivo (VULNERABLE)
        File requestedFile = new File(BASE_DIR + filename);

        // 3. Intento de Ataque:
        // Petición: /app/download?filename=../../../../etc/passwd
        // Ruta construida: /var/www/data/docs/../../../../etc/passwd
        // Esto resuelve a /etc/passwd en el sistema operativo, revelando información sensible.

        if (requestedFile.exists() && requestedFile.isFile()) {
            // Lógica para servir el archivo...
        } else {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
        }
    }
}
```

Figura 13 *Path traversal.* Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Un atacante podría enviar una solicitud con un nombre de archivo manipulado, como:

- Petición: /app/download?filename=../../../../etc/passwd
- Ruta construida: /var/www/data/docs/../../../../etc/passwd
- Esto resuelve a /etc/passwd en el sistema operativo y revela información.

El sistema de archivos interpretaría .. como un **retroceso en el directorio,** lo que permitiría al atacante acceder a **archivos sensibles,** como /etc/passwd (que contiene información de los usuarios del sistema en Linux), fuera del directorio de carga.

Solución

Solución segura: **lista blanca** y **normalización de ruta.** La defensa más robusta implementa dos pasos de validación:

- **Lista blanca:** solo permite el acceso a un conjunto predefinido de nombres de archivo.
- **Normalización:** usa rutas canónicas para garantizar que, incluso si se usa una ruta relativa, el archivo final esté contenido en el directorio base.

Resumen de la solución:

- **Lista blanca (ALLOWED_FILES.contains(filename)):** el método de defensa más simple y robusto. Solo permite nombres de archivo que coincidan exactamente con un conjunto predefinido de valores seguros.
- **Normalización (requestedPath.normalize()):** la función clave es normalize(), que procesa y elimina secuencias, como .., y resuelve la ruta relativa.

© Universidad Internacional de La Rioja (UNIR)

- **Verificación de prefijo (.startsWith()):** se utiliza la ruta normalizada y absoluta para verificar que comience con la ruta base absoluta (absoluteBasePath). Si el atacante introduce ../../etc/passwd, la ruta normalizada y absoluta resultante no comenzará con /var/www/data/docs/ y se bloqueará el ataque.

```java
import java.io.IOException;
import java.nio.file.*;
import java.util.Set;
import javax.servlet.http.*;

public class ShortSafeDownloadServlet extends HttpServlet {

    // Directorio base seguro, definido con la API Path
    private static final Path BASE_PATH = Paths.get("/var/www/data/docs/");

    // Lista Blanca: Solo estos nombres de archivo están permitidos.
    private static final Set<String> ALLOWED_FILES = Set.of(
        "factura_2025.pdf", "manual_guia.html", "config_segura.ini"
    );

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String filename = request.getParameter("file");

        // 1. **DEFENSA: LISTA BLANCA**
        if (filename == null || !ALLOWED_FILES.contains(filename)) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Archivo no autorizado o inexistente.");
            return;
        }

        // 2. Construir la ruta combinando el directorio base y la entrada validada
        Path requestedPath = BASE_PATH.resolve(filename);

        // 3. **DEFENSA: NORMALIZACIÓN Y VERIFICACIÓN**
        // a) .normalize(): Resuelve componentes de ruta como '..' y '.'
        // b) .toAbsolutePath(): Convierte a una ruta absoluta
        // c) .startsWith(): Asegura que la ruta final esté dentro del BASE_PATH absoluto

        Path normalizedPath = requestedPath.normalize().toAbsolutePath();
        Path absoluteBasePath = BASE_PATH.toAbsolutePath(); // Ruta base absoluta para la verificación

        if (!normalizedPath.startsWith(absoluteBasePath)) {
            // Esto atrapa cualquier intento sofisticado de escapar del directorio base.
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Path Traversal detectado.");
            return;
        }

        // 4. Si las defensas pasan, es seguro acceder al archivo
        // Nota: Se recomienda toRealPath(LinkOption.NOFOLLOW_LINKS) para seguridad total contra symlinks.
        // Pero para un ejemplo conciso, la normalización absoluta es una gran mejora.

        if (Files.exists(normalizedPath) && Files.isRegularFile(normalizedPath)) {
            // Lógica para servir el archivo (p. ej., Files.copy)
            response.setStatus(HttpServletResponse.SC_OK);
            // ...
        } else {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
        }
    }
}
```

Figura 14. Solución *path traversal.* Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Command injection SO CWE 78

La vulnerabilidad de **Command injection** (inyección de comandos) ocurre cuando una aplicación ejecuta comandos del sistema operativo (SO) utilizando una **entrada de usuario no validada** como parte de la cadena de comando. Un atacante puede inyectar comandos adicionales que se ejecutarán en el servidor.

Imaginar un Servlet diseñado para ejecutar una utilidad de red, como **ping o nslookup**, donde el nombre de *host* (servidor) es proporcionado por el usuario a través de un parámetro.

```java
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.BufferedReader;
import javax.servlet.http.*;

public class VulnerableServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String host = request.getParameter("host");

        // ❌ PUNTO VULNERABLE: La entrada del usuario se concatena directamente en el comando.
        String command = "ping -c 4 " + host;

        try {
            // ❌ PELIGRO: Runtime.exec(String) invoca el shell del SO, permitiendo la inyección
            Process p = Runtime.getRuntime().exec(command);

            // Lógica para leer la salida...
            BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                response.getWriter().println(line);
            }
        } catch (Exception e) {
            response.sendError(500);
        }
    }
}
// Ataque: ?host=127.0.0.1; cat /etc/passwd
// Comando ejecutado: ping -c 4 127.0.0.1; cat /etc/passwd
```

Figura 15. Command injection. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Explotación

Un atacante puede utilizar meta caracteres del *shell* (como ;, &&, |) para **encadenar comandos.**

- **Entrada del atacante para el parámetro *host:*** 127.0.0.1; cat /etc/passwd
- **Comando final ejecutado en el SO:** ping -c 4 127.0.0.1; cat /etc/passwd

El servidor primero ejecuta el ping e, inmediatamente después, ejecuta el comando inyectado cat /etc/passwd, para revelar el archivo de **contraseñas** del sistema (si el usuario que ejecuta el proceso tiene permisos de lectura).

Solución:

La solución más efectiva es **evitar la ejecución de comandos del SO** si es posible. Si es absolutamente necesario, se debe:

- **Utilizar la sintaxis de *array* para Runtime.exec() o ProcessBuilder:** esta sintaxis evita que la entrada del usuario sea interpretada por el *shell* del sistema operativo.
- **Validación y saneamiento rigurosos (lista blanca):** si se requiere la entrada del usuario, solo se deben permitir caracteres y formatos de entrada seguros.

Se usa ProcessBuilder con un *array* de *strings*, lo cual asegura que la entrada del usuario sea tratada como un **argumento único** para el comando principal, no como un comando separado.

© Universidad Internacional de La Rioja (UNIR)

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.regex.Pattern;
import javax.servlet.http.*;

public class SafeServlet extends HttpServlet {
    // ✅ DEFENSA 1: Lista Blanca para la entrada
    private static final Pattern SAFE_HOST_PATTERN = Pattern.compile("^[a-zA-Z0-9\\.\\-]+$");

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String host = request.getParameter("host");

        // 1. Validación estricta
        if (host == null || !SAFE_HOST_PATTERN.matcher(host).matches()) {
            response.sendError(400, "Host inválido.");
            return;
        }

        // 2. ✅ DEFENSA 2: Uso de ProcessBuilder con array de argumentos
        // Los comandos maliciosos inyectados son tratados como parte del argumento 'host', NO como comandos separados.
        ProcessBuilder pb = new ProcessBuilder("ping", "-c", "4", host);

        try {
            Process p = pb.start();
            p.waitFor();

            // Lógica para leer la salida de forma segura...
            try (BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream(), StandardCharsets.UTF_8))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    response.getWriter().println(line);
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            response.sendError(500);
        } catch (IOException e) {
            response.sendError(500, "Error de ejecución.");
        }
    }
}
```

Figura 16. Solución command injection. Fuente: elaboración propia.

Principios de la solución:

- **Evitar el *shell* (defensa principal):** al usar ProcessBuilder (o Runtime.exec(String[])) y pasar cada argumento del comando en un *array*, se invoca el ejecutable directamente sin la intermediación de un *shell*. Si el atacante introduce 127.0.0.1; cat /etc/passwd, el comando ejecutado será, literalmente, ping -c 4 "127.0.0.1; cat /etc/passwd". El programa ping no sabrá qué hacer con ese argumento y fallará, pero no ejecutará el comando malicioso.

© Universidad Internacional de La Rioja (UNIR)

- **Lista blanca (defensa secundaria):** la validación estricta (SAFE_HOST_PATTERN) es una capa de defensa crucial. Asegura que la entrada solo contenga los caracteres esperados (letras, números, puntos, guiones) y rechaza inmediatamente los metacaracteres del *shell* (&, |, ;, $, etc.) antes de que lleguen a la ejecución.

Missing authorization CWE 862

La vulnerabilidad Missing authorization, clasificada como CWE-862, ocurre cuando un componente de la aplicación no implementa o implementa de forma **incorrecta** las **comprobaciones** necesarias para determinar si un usuario (que ya ha sido autenticado o identificado) tiene el **permiso** necesario para acceder a un recurso o realizar una acción. Imagina un Java Servlet que maneja la eliminación de cuentas de usuario. Este Servlet comprueba si el usuario está conectado (autenticado), pero **nunca verifica si el usuario es un administrador**.

```
// Servlet para una función sensible, e.g., /admin/resetSystem
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException {

    HttpSession session = request.getSession(false);

    // ❌ 1. Sólo verifica la Autenticación (¿Está logueado?)
    if (session == null || session.getAttribute("userId") == null) {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED); // 401
        return;
    }

    // ❌ PUNTO VULNERABLE: No hay verificación de si el 'userId' tiene rol de ADMIN.
    // ... Código sensible que ejecuta la acción (e.g., resetSystem()) ...
    response.getWriter().println("Sistema reiniciado con éxito.");
}
// Un usuario estándar puede acceder a esta URL y ejecutar la acción.
```

Figura 17. CWE 862. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Explotación:

- Un usuario estándar (no administrador) se autentica e **inicia sesión.**
- El usuario estándar envía una **solicitud POST** directamente a la URL: POST /app/admin/deleteUser?id=12345
- El Servlet solo encuentra la **sesión válida** (punto 1).
- Debido a la Missing authorization, el Servlet ejecuta la **eliminación** (punto 2), lo que permite a un usuario normal realizar una **acción de administrador.**

Solución: implementar verificación de rol (autorización).

La solución consiste en añadir una **comprobación explícita** después de la autenticación para determinar el rol o **permiso** del usuario y denegar el acceso si no cumple con el requisito. Se asume que el rol del usuario se almacena en la sesión después de la autenticación ("userRole": "ADMIN" o "userRole": "STANDARD").

```
// Servlet para una función sensible, e.g., /admin/resetSystem
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException {

    HttpSession session = request.getSession(false);

    // 1. Verificar Autenticación
    if (session == null || session.getAttribute("userId") == null) {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
        return;
    }

    // ✅ DEFENSA: Implementar Autorización (Verificación de Rol)
    String userRole = (String) session.getAttribute("userRole");

    // 2. Verificar que el usuario tenga el rol requerido ("ADMIN")
    if (!"ADMIN".equals(userRole)) {
        // Se niega el acceso a usuarios que no tienen los permisos necesarios.
        response.sendError(HttpServletResponse.SC_FORBIDDEN, "Acceso denegado. Se
requiere rol de Admin."); // 403
        return;
    }

    // ... Código seguro que ejecuta la acción (sólo para administradores) ...
    response.getWriter().println("Sistema reiniciado con éxito.");
}
```

Figura 18. Comprobación de rol. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Principios de **prevención:**

La regla fundamental para prevenir CWE-862 es **denegar por defecto** *(fail-safe defaults):*

- **Autenticación:** asegúrate de que el usuario esté identificado.
- **Autorización (*check* de rol/permiso):** después de la autenticación, comprueba el rol o permiso del usuario (userRole en la sesión) contra un valor de lista blanca (REQUIRED_ROLE).
- **Acceso:** el acceso a la funcionalidad solo se concede si la comprobación de autorización es exitosa. Si falla, se devuelve un código 403 Forbidden.

En entornos Java EE/Jakarta EE, esta verificación de rol a menudo se maneja de manera más elegante y global utilizando el archivo de **configuración de seguridad** (web.xml) o anotaciones, como @RolesAllowed o @ServletSecurity, lo cual **externaliza** la **lógica de autorización** del código del Servlet.

Unrestricted upload of file with dangerous type CWE-434

La vulnerabilidad Unrestricted upload of file with dangerous type (CWE-434) ocurre cuando una aplicación permite a un usuario subir **archivos de cualquier tipo** (como un *script* ejecutable o un *webshell*) **sin validar** o **sanear** el contenido o la extensión y luego permite que ese archivo sea accesible y **ejecutado** por el servidor web.

Ejemplo de **código Java vulnerable:** este Servlet acepta cualquier archivo subido y lo guarda directamente en un directorio accesible públicamente dentro del servidor web, sin verificar la extensión o el tipo de contenido.

© Universidad Internacional de La Rioja (UNIR)

```java
import java.io.*;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.*;

// Permite subir archivos multipartes
@MultipartConfig(fileSizeThreshold = 1024 * 1024 * 1, // 1MB
                 maxFileSize = 1024 * 1024 * 10,      // 10MB
                 maxRequestSize = 1024 * 1024 * 50)   // 50MB
public class VulnerableUploadServlet extends HttpServlet {

    // ❌ Directorio accesible por el servidor web (e.g., dentro de /webapp)
    private static final String UPLOAD_DIR = "/var/www/html/uploads/";

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        // ... Lógica de autenticación/autorización omitida por brevedad ...

        try {
            // 1. Obtener la parte del archivo
            Part filePart = request.getPart("file");
            String fileName = filePart.getSubmittedFileName();

            // ❌ PUNTO VULNERABLE: No hay validación de la extensión o del contenido
            File file = new File(UPLOAD_DIR + fileName);

            // 2. Escribir el archivo sin ninguna restricción
            filePart.write(file.getAbsolutePath());

            response.getWriter().println("Archivo subido con éxito a: " + fileName);

        } catch (Exception e) {
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "Error al
subir el archivo.");
        }
    }
}
```

Figura 19. CWE 434. Fuente: elaboración propia.

Escenario de explotación:

- Un atacante crea un archivo llamado shell.jsp que contiene código Java (un *webshell*) que le permite ejecutar comandos del sistema operativo.
- El atacante sube shell.jsp a través del VulnerableUploadServlet. El Servlet lo guarda como /var/www/html/uploads/shell.jsp.
- El atacante accede directamente al archivo a través del navegador: http://servidor.com/uploads/shell.jsp.
- El servidor web (Tomcat, por ejemplo) procesa el archivo .jsp y ejecuta el código malicioso, otorgando al atacante control remoto sobre el servidor.

© Universidad Internacional de La Rioja (UNIR)

Solución: validación estricta del archivo. La solución se basa en tres pilares: validación de lista blanca, validación de *content-type* y almacenamiento seguro fuera de la raíz web.

Principios de prevención (defensa por capas):

- **Validación de extensión (lista blanca):** es la defensa principal. Solo se permiten extensiones conocidas y seguras (.jpg, .pdf, etc.). Se debe prohibir explícitamente cualquier extensión que el servidor pueda ejecutar (.jsp, .php, .exe, etc.).
- **Validación de contenido (MIME *type* y contenido real):** aunque no está en el código corto, es vital verificar el MIME *type* del encabezado y, si es posible, el *magic byte* del archivo para confirmar que el contenido coincide con la extensión declarada (para evitar que un atacante renombre un shell.jsp a shell.jpg).
- Para imágenes, se recomienda **inspeccionar** los **primeros bytes** *(magic bytes)* o reprocesar/reescribir la imagen para eliminar metadatos o código incrustado.
- **Almacenamiento seguro:**
  - **Fuera de la raíz web:** guardar los archivos en un directorio que no sea accesible directamente a través de una URL (/opt/appdata/ en lugar de /var/www/html/). Si un archivo ejecutable se sube, el servidor web nunca podrá procesarlo.
  - **Renombrado:** renombrar el archivo subido (usando UUIDs o un *timestamp*) para evitar que el atacante intente usar nombres especiales o inyecciones de caracteres en el nombre.

© Universidad Internacional de La Rioja (UNIR)

```java
import java.io.IOException;
import java.io.File;
import java.util.Set;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.*;

@MultipartConfig // (Configuración omitida por brevedad)
public class SafeUploadServlet extends HttpServlet {

    // ✅ Directorio NO accesible por el servidor web (fuera de la raíz web)
    private static final String SAFE_STORAGE_DIR = "/opt/appdata/useruploads/";

    // ✅ DEFENSA 1: Lista Blanca de extensiones permitidas
    private static final Set<String> ALLOWED_EXTENSIONS = Set.of("jpg", "jpeg", "png",
"pdf", "docx");

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        try {
            Part filePart = request.getPart("file");
            String fileName = filePart.getSubmittedFileName();

            // Asegurarse de que el nombre del archivo no contenga path traversal
            // Aunque no es el enfoque principal de este CWE, siempre es necesario.
            fileName = sanitizeFilename(fileName);

            // 1. Extraer la extensión
            String fileExtension = getFileExtension(fileName);

            // 2. ✅ DEFENSA 2: Validar extensión contra la Lista Blanca
            if (!ALLOWED_EXTENSIONS.contains(fileExtension)) {
                response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Tipo de archivo
no permitido: ." + fileExtension);
                return;
            }

            // (Opcional, pero recomendado: Verificar el Content-Type (MIME type)
            // String contentType = filePart.getContentType();
            // if (!contentType.startsWith("image/") &&
!contentType.equals("application/pdf")) { ... }

            // 3. ✅ DEFENSA 3: Almacenamiento seguro y renombrado
            // Renombrar el archivo o usar un UUID para evitar conflictos y ataques de
extensión múltiple.
            String newFileName = System.currentTimeMillis() + "." + fileExtension;
            File file = new File(SAFE_STORAGE_DIR + newFileName);

            filePart.write(file.getAbsolutePath());

            response.getWriter().println("Archivo subido y renombrado con éxito: " +
newFileName);

        } catch (Exception e) {
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "Error al
subir el archivo.");
        }
    }

    // Métodos auxiliares simples
    private String getFileExtension(String fileName) {
        int dotIndex = fileName.lastIndexOf('.');
        return (dotIndex == -1) ? "" : fileName.substring(dotIndex + 1).toLowerCase();
    }

    private String sanitizeFilename(String fileName) {
        // Elimina caracteres de path traversal (../) y asegura que sea solo el nombre.
        return fileName.replaceAll("[^a-zA-Z0-9\\.\\-_]", "_");
    }
}
```

Figura 20. Solución CWE 434. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Improper restriction of XML external entity reference CWE-611

La vulnerabilidad Improper restriction of XML external entity reference (CWE-611), comúnmente conocida como XXE injection, ocurre cuando un *parser* de XML procesa una **entrada XML** que contiene referencias a **entidades externas** definidas por el usuario (DTDs). Un atacante puede explotar esto para obligar al servidor a divulgar archivos locales, realizar peticiones de red internas (SSRF) o, incluso, realizar un ataque de denegación de servicio (DoS) (Mouse, 2019).

Ejemplo de código vulnerable (XXE). Este Servlet utiliza la librería estándar de Java para *parsear* documentos XML, lo que, por defecto, permite el procesamiento de entidades externas.

```
import java.io.*;
import javax.servlet.http.*;
import javax.xml.parsers.*;
import org.w3c.dom.Document;

public class VulnerableXMLServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        try {
            // 1. Crear el parser (Default settings son vulnerables a XXE)
            DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
            DocumentBuilder db = dbf.newDocumentBuilder();

            // ❌ PUNTO VULNERABLE: El parser acepta entrada XML con entidades externas.
            Document doc = db.parse(request.getInputStream());

            // 2. Procesar el documento (la vulnerabilidad ocurre en la línea anterior)
            String elementValue =
doc.getElementsByTagName("user").item(0).getTextContent();

            response.getWriter().println("Valor procesado: " + elementValue);

        } catch (Exception e) {
            response.sendError(500, "Error al procesar XML: " + e.getMessage());
        }
    }
}
```

Figura 21. CWE 611. Fuente: elaboración propia.

Carga útil de explotación (XML malicioso). El atacante envía un cuerpo XML que define una entidad externa (&xxe;) que apunta a un **archivo sensible** (/etc/passwd).

© Universidad Internacional de La Rioja (UNIR)

```
1  <?xml version="1.0" encoding="ISO-8859-1"?>
2  <!DOCTYPE foo [
3    <!ELEMENT foo ANY >
4    <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
5  <userRequest>
6      <username>&xxe;</username>
7      <password>dummy</password>
8  </userRequest>
```

Figura 22. Carga útil XXE. Fuente: elaboración propia.

Resultado del ataque: si el *parser* es vulnerable, al intentar resolver la entidad &xxe;, leerá el contenido de /etc/passwd y lo insertará como el valor del tag <username>. La respuesta del servidor podría revelar el **contenido del archivo:**

Procesado XML para usuario: root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

...

Solución segura en Java. La solución consiste en **desactivar** explícitamente el procesamiento de DTDs y entidades externas en la configuración del DocumentBuilderFactory antes de crear el *parser.* Esto sigue las recomendaciones de la *OWASP XXE Prevention Cheat Sheet* (s. f.).

L a **prevención** de XXE se basa en la configuración del *parser*. El enfoque de **denegar por defecto** requiere que deshabilites las funciones que permiten la inclusión de contenido externo:

- **Deshabilitar DTDs:** desactivar la declaración de tipo de documento (DOCTYPE).
- **Deshabilitar entidades externas:** desactivar explícitamente el procesamiento de entidades generales y de parámetros externos.

© Universidad Internacional de La Rioja (UNIR)

La solución más fuerte es usar la opción http://apache.org/xml/features/disallow-doctype-decl, ya que **bloquea** el **mecanismo de ataque** más común en la raíz.

```java
import java.io.IOException;
import javax.servlet.http.*;
import javax.xml.parsers.*;
import org.w3c.dom.Document;

public class SafeXMLServlet extends HttpServlet {
    protected void void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        try {
            DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

            // ✅ DEFENSA: Deshabilitar el procesamiento de DTDs y entidades externas.
            // Estas dos líneas son la solución estándar para mitigar XXE.
            dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
// Deshabilita DTDs completamente
            dbf.setFeature("http://xml.org/sax/features/external-general-entities",
false);
            dbf.setFeature("http://xml.org/sax/features/external-parameter-entities",
false);

            DocumentBuilder db = dbf.newDocumentBuilder();

            // ❌ Ahora, si el XML contiene entidades externas, el parser lanzará una
excepción.
            Document doc = db.parse(request.getInputStream());

            String elementValue =
doc.getElementsByTagName("user").item(0).getTextContent();

            response.getWriter().println("Valor procesado: " + elementValue);

        } catch (ParserConfigurationException e) {
             response.sendError(500, "Error de configuración del parser.");
        } catch (Exception e) {
            response.sendError(400, "XML inválido o ataque detectado.");
        }
    }
}
```

Figura 23. Solución XXE. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Server-side request forgery (SSRF) CWE-918

La vulnerabilidad Server-side request forgery (SSRF), clasificada como CWE-918, ocurre cuando una aplicación web procesa una **URL** proporcionada por el **usuario** y la utiliza para realizar una **solicitud de red** (HTTP, FTP, etc.) sin validación adecuada. Un atacante puede explotar esto para hacer que el servidor realice solicitudes a **recursos internos** (como la *intranet*, servicios en la nube o *localhost*) a los que el atacante no debería tener acceso. Este Servlet lee una URL del usuario y realiza una solicitud GET a esa URL para **obtener datos,** lo cual es un patrón común y vulnerable a SSRF.



```java
import java.io.IOException;
import java.net.*;
import javax.servlet.http.*;

public class VulnerableSSRFServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String externalUrl = request.getParameter("url");
        if (externalUrl == null) { return; }

        try {
            // ❌ VULNERABLE: Permite al servidor conectarse a cualquier host, incluso internos.
            URL url = new URL(externalUrl);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            // ... Código para leer y devolver la respuesta ...
            conn.disconnect();
        } catch (Exception e) { response.sendError(500); }
    }
}
// Ataque: ?url=http://127.0.0.1/admin (Acceso a localhost)
```

Figura 24. CWE 918. Fuente: elaboración propia.

Vector de ataque: el atacante puede usar esta vulnerabilidad para escanear y acceder a la red interna del servidor. Payload de ataque (para intentar acceder al panel de administración local): http://servidor.com/getdata?url=http://127.0.0.1/admin

O para acceder a **metadatos sensibles** de un entorno de nube (AWS, Azure, etc.): http://servidor.com/getdata?url=http://169.254.169.254/latest/meta-data/

© Universidad Internacional de La Rioja (UNIR)

Solución: la prevención de SSRF requiere aplicar una lista blanca estricta para las URL, protocolos y *hosts* permitidos y, al mismo tiempo, bloquear direcciones de red internas (como *localhost* o direcciones IP privadas).

```java
import java.io.IOException;
import java.net.*;
import java.util.Set;
import javax.servlet.http.*;

public class SafeSSRFServlet extends HttpServlet {

    // ✅ Lista Blanca de hosts aprobados
    private static final Set<String> ALLOWED_DOMAINS = Set.of("api.externa.com");

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String externalUrl = request.getParameter("url");
        if (externalUrl == null) { return; }

        try {
            URL url = new URL(externalUrl);
            String host = url.getHost().toLowerCase();

            // ✅ DEFENSA 1: Lista Blanca (Solo hosts externos conocidos)
            if (!ALLOWED_DOMAINS.contains(host)) {
                // ✅ DEFENSA 2: Bloqueo de IP y Localhost (Lista Negra)
                if (host.equals("127.0.0.1") || host.equals("localhost") ||
host.startsWith("192.168.")) {
                    response.sendError(HttpServletResponse.SC_FORBIDDEN, "Acceso interno
denegado.");
                    return;
                }
                response.sendError(HttpServletResponse.SC_FORBIDDEN, "Host no
autorizado.");
                return;
            }

            // ... Conexión segura si la validación pasó ...
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            // ...
            conn.disconnect();
        } catch (Exception e) { response.sendError(500); }
    }
}
```

Figura 25. Solución CWE 918. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Use of hard-coded credentials CWE-798

La vulnerabilidad CWE-798: uso de credenciales *hard*-codeadas ocurre cuando un desarrollador incrusta **información sensible** de **autenticación** (como contraseñas, claves API o secretos criptográficos) directamente en el **código fuente** de la aplicación. Esto es un riesgo grave porque cualquiera que obtenga acceso al código (por ejemplo, a través de un repositorio público, una fuga o descompilando el *bytecode* de Java) puede obtener las credenciales y comprometer sistemas externos.

Ejemplo de código Java vulnerable (CWE-798). Este Servlet se conecta a una base de datos usando un usuario y una contraseña que están **escritos** directamente en el **código**:

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import javax.servlet.http.HttpServlet;

public class VulnerableDBServlet extends HttpServlet {

    protected void connectToDatabase() {
        try {
            // ❌ VULNERABLE: Las credenciales están incrustadas en el código.
            String dbUser = "admin";
            String dbPassword = "Password123!"; // ¡Contraseña a la vista!
            String dbUrl = "jdbc:mysql://localhost:3306/mydatabase";

            Connection conn = DriverManager.getConnection(dbUrl, dbUser, dbPassword);

            // ... Lógica para usar la conexión ...

            conn.close();
        } catch (SQLException e) {
            // Manejo de errores...
        }
    }
}
```

Figura 26. CWE 798. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Solución segura: la solución es **externalizar** las **credenciales** cargándolas desde una fuente segura en tiempo de ejecución, como variables de entorno o un archivo de propiedades fuera del código fuente.

Externalización: las credenciales (DB_USER, DB_PASSWORD) ya no están en el código.

Configuración del entorno: en el servidor donde se despliega la aplicación, se configuran estas variables de entorno. Por ejemplo, en Linux:

```
#Bash
#export DB_USER="admin_prod"
#export DB_PASSWORD="UnaContraseñaMuySeguraYCompleja"
#export DB_URL="jdbc:mysql://prod-db-server:3306/productiondb"
```

Seguridad: el código fuente ya no contiene secretos. La gestión de las credenciales se traslada al entorno de despliegue, que es mucho más seguro y controlado.

© Universidad Internacional de La Rioja (UNIR)

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import javax.servlet.http.HttpServlet;

public class SafeDBServlet extends HttpServlet {

    protected void connectToDatabase() {
        try {
            // ✅ SEGURO: Las credenciales se leen desde variables de entorno.
            String dbUser = System.getenv("DB_USER");
            String dbPassword = System.getenv("DB_PASSWORD");
            String dbUrl = System.getenv("DB_URL");

            // Si las variables no están definidas, el sistema falla de forma segura.
            if (dbUser == null || dbPassword == null || dbUrl == null) {
                throw new SQLException("Faltan credenciales de la base de datos en la configuración.");
            }

            Connection conn = DriverManager.getConnection(dbUrl, dbUser, dbPassword);

            // ... Lógica para usar la conexión ...

            conn.close();
        } catch (SQLException e) {
            // Manejo de errores...
        }
    }
}
```

Figura 27. Solución CWE 798. Fuente: elaboración propia.

Uncontrolled resource consumption CWE-400

La vulnerabilidad CWE-400: consumo incontrolado de recursos (uncontrolled resource consumption) ocurre cuando una aplicación **no limita** adecuadamente la **cantidad** o el **tamaño** de los recursos que puede asignar o utilizar, permitiendo que un atacante consuma recursos excesivos, lo que generalmente lleva a una **denegación de servicio** (DoS) por agotamiento de recursos (memoria, CPU, conexiones, etc.). Aquí, se muestra un ejemplo corto de esta vulnerabilidad en un Java Servlet y su solución, centrado en el **agotamiento de memoria/tiempo** de CPU a través de un procesamiento excesivo de datos.

Este Servlet acepta un parámetro *length* y utiliza ese valor para crear y procesar una lista de ese tamaño. Si el atacante proporciona un valor muy grande, agotará la memoria del servidor (OutOfMemoryError) o el tiempo de CPU.

© Universidad Internacional de La Rioja (UNIR)

```java
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.http.*;

public class VulnerableLoopServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String lengthStr = request.getParameter("length");

        if (lengthStr == null) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST);
            return;
        }

        try {
            int length = Integer.parseInt(lengthStr);

            // ❌ VULNERABLE: No hay límite en el tamaño del loop/lista.
            // Si length es 2,000,000,000, el servidor puede colapsar.
            List<String> largeList = new ArrayList<>();

            for (int i = 0; i < length; i++) {
                largeList.add("Data" + i);
            }

            response.getWriter().println("Proceso completado para " + length + "
elementos.");

        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Formato de longitud
inválido.");
        } catch (OutOfMemoryError e) {
            // Esto es lo que el atacante busca
            response.sendError(HttpServletResponse.SC_SERVICE_UNAVAILABLE, "Servidor
sobrecargado.");
        }
    }
}
// Ataque: /process?length=2000000000 (Consume gigabytes de memoria hasta el fallo).
```

Figura 28. CWE 400. Fuente: elaboración propia.

Ejemplo de *payload* de ataque:

https://sitioweb/process?length=2000000000 (Consume gigabytes de memoria hasta el fallo).

Solución: la solución principal es comprobar el **tamaño** de la **entrada** de **datos** y no permitir aquellos que excedan el máximo permitido:

© Universidad Internacional de La Rioja (UNIR)

```java
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.http.*;

public class SafeLoopServlet extends HttpServlet {

    // ✅ DEFENSA: Límite máximo estricto definido por la aplicación.
    private static final int MAX_LENGTH = 100000;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String lengthStr = request.getParameter("length");

        if (lengthStr == null) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST);
            return;
        }

        try {
            int length = Integer.parseInt(lengthStr);

            // ✅ DEFENSA: Controlar y restringir el valor de la entrada
            if (length > MAX_LENGTH) {
                response.sendError(HttpServletResponse.SC_BAD_REQUEST,
                    "La longitud solicitada excede el límite de seguridad de " +
MAX_LENGTH);
                return;
            }

            // Procesamiento seguro con un límite conocido
            List<String> safeList = new ArrayList<>();
            for (int i = 0; i < length; i++) {
                safeList.add("Data" + i);
            }

            response.getWriter().println("Proceso completado para " + length + "
elementos.");

        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Formato de longitud
inválido.");
        }
    }
}
```

Figura 29. Solución CWE 400. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

HTTP response splitting CWE 113

La vulnerabilidad HTTP request/response splitting (CWE-113) surge cuando una aplicación web permite que un atacante inyecte caracteres de **salto de línea** y **retorno de carro** (\r\n o %0d%0a) en una **respuesta HTTP,** típicamente dentro de un encabezado HTTP. Esto puede permitir al atacante dividir la respuesta, inyectando encabezados HTTP adicionales e, incluso, un cuerpo de respuesta completamente nuevo, llevando a ataques como *cross-site scripting* (XSS) o envenenamiento de caché *(cache poisoning)*.

Ejemplo de vulnerabilidad (código vulnerable): este Servlet utiliza entrada de usuario no saneada para establecer un valor de *cookie,* que es parte del encabezado de la respuesta HTTP.

```java
import java.io.IOException;
import javax.servlet.http.*;

public class VulnerableHeaderServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        // 1. Obtener entrada del usuario (NO VALIDADA)
        String userData = request.getParameter("user_setting");

        if (userData == null) { return; }

        // ❌ PUNTO VULNERABLE: userData se inserta directamente en el encabezado Set-Cookie.
        // Si userData contiene %0d%0a, se inyectan nuevos encabezados.
        Cookie cookie = new Cookie("user_pref", userData);
        response.addCookie(cookie);

        response.getWriter().println("Preferencia de usuario guardada.");
    }
}
```

Figura 30. Solución CWE 400. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Explotación: un atacante puede enviar un *payload* que contiene caracteres de salto de línea codificados en URL, *payload* para user_setting:

```
http://sitioweb?user_setting=validvalue%0d%0aX-Injected-Header: Injected%0d%0aContent-Length: 0%0d%0a%0d%0aHTTP/1.1 200 OK%0d%0aContent-Type: text/html%0d%0a%0d%0a<script>alert(1)</script>
```

Figura 31. Ejemplo de *payload* de ataque CWE 400. Fuente: elaboración propia.

```
HTTP/1.1 200 OK
Set-Cookie: user_pref=validvalue
X-Injected-Header: Injected
Content-Length: 0

HTTP/1.1 200 OK  <-- ¡El atacante inyectó el inicio de una nueva respuesta!
Content-Type: text/html

<script>alert(1)</script>
```

Figura 32. Ejemplo de duplicidad de respuestas CWE 400. Fuente: elaboración propia.

```
import java.io.IOException;
import javax.servlet.http.*;

public class SafeHeaderServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String userData = request.getParameter("user_setting");
        if (userData == null) { return; }

        // ✅ DEFENSA: Saneamiento de la entrada
        // Eliminar explícitamente los caracteres de retorno de carro (%0d) y salto de línea (%0a).
        String safeUserData = userData
            .replace('\n', ' ') // Salto de línea (LF)
            .replace('\r', ' '); // Retorno de carro (CR)

        // Nota: También se puede utilizar la codificación URL o la codificación específica de la cabecera.

        // ✅ Uso seguro de la entrada saneada
        // La implementación de Servlet/Contenedor moderno (Tomcat/Jetty) a menudo hace esto automáticamente
        // si se utiliza response.addCookie() o response.setHeader(), pero la limpieza explícita es crucial.
        Cookie cookie = new Cookie("user_pref", safeUserData);
        response.addCookie(cookie);

        response.getWriter().println("Preferencia de usuario guardada.");
    }
}
```


© Universidad Internacional de La Rioja (UNIR)

Figura 33. Solución CWE 400. Fuente: elaboración propia.

```java
import java.io.IOException;
import javax.servlet.http.*;
import org.owasp.encoder.Encode; // Importar OWASP Encoder

public class SafeHeaderServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        String userData = request.getParameter("user_setting");
        if (userData == null) { return; }

        // ✅ DEFENSA: Codificar la entrada del usuario antes de insertarla en el encabezado.
        // Usa forCookie() si la biblioteca lo soporta, o forJava() como opción general de codificación estricta.
        // Algunos contenedores de servlets ya sanitizan las cookies, pero la codificación explícita es más segura.
        String safeUserData = Encode.forJava(userData);

        // Si la biblioteca soporta codificación específica de cookies:
        // String safeUserData = Encode.forCookie(userData);

        // ✅ Uso seguro de la entrada codificada.
        Cookie cookie = new Cookie("user_pref", safeUserData);
        response.addCookie(cookie);

        response.getWriter().println("Preferencia de usuario guardada.");
    }
}
```

Figura 34. Solución OWASP Java *encoder* CWE 400. Fuente: elaboración propia.

Principios de prevención. La prevención de CWE-113 se basa en:

- **Eliminación o codificación:** eliminar o reemplazar los caracteres de CR (\r o %0d) y LF (\n o %0a) de cualquier entrada de usuario que se inserte en un encabezado de respuesta HTTP.
- **Uso de librerías de codificación segura:** uso de librerías específicas de codificación de la salida, como OWASP Java Encoder.

© Universidad Internacional de La Rioja (UNIR)

6.4. Referencias bibliográficas

Bermejo, J. R. (2021). *OWASP Top Ten Benchmark.* GitHub. https://github.com/jrbermh/OWASP-Top-Ten-Benchmark

Chess, B. y West, J. (2007). *Secure programming with static analysis*. Addison-Wesley.

Debilidades. (s. f.). https://vulncat.fortify.com/es/weakness

Graff, M. y Van Wyk, K. R. (2003). *Secure coding: principles and practices.* O'Reilly Media. https://www.veracode.com/state-of-software-security-report

Hoffman, A. (2024) *Web application security* (2ª ed.). O'Reilly Media.

https://ptgmedia.pearsoncmg.com/images/9780321424778/samplepages/9780321424778.pdf

McDonald, M. (2024) *Grokking web application security.* Simon & Schuster.

McGraw, G. (2006). *Software security: building security.* Addison- Wesley Professional.

Mouse, J. C. (2019). *Validar documentos XML con esquema XSD.* JC Mouse. https://www.jc-mouse.net/java/validar-documentos-xml-con-esquema-xsd

OWASP SQL Injection Prevention Cheat Sheet. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Predictable pseudorandom number generator. (s. f.). https://find-sec-bugs.github.io/bugs.htm

Static code analysis tools for your Java. (s. f.). https://www.sonarsource.com/knowledge/languages/java/

© Universidad Internacional de La Rioja (UNIR)

Vulnerabilidades en el código fuente: sitio web de Fortify

Debilidades. (s. f.). https://vulncat.fortify.com/es/weakness

El sitio web de Fortify describe las vulnerabilidades web más frecuentes y peligrosas en el código fuente con ejemplos.

© Universidad Internacional de La Rioja (UNIR)

Seguridad en frameworks

Alex, B., Taylor, L., Winch, R., Hillert, G., Grandja, J., Bryant, J., Meléndez, E., Cummings, J. y Syer, D. (s. f.). *Spring security reference.* https://docs.spring.io/spring-security/site/docs/5.4.0/reference/pdf/spring-security-reference.pdf

Se profundiza en diversas librerías de validación en el código fuente de diversos *frameworks* de desarrollo de aplicaciones web.

© Universidad Internacional de La Rioja (UNIR)

Vulnerabilidades en el código fuente: sitio web de Find Security Bugs

Predictable pseudorandom number generator. (s. f.). https://find-sec-bugs.github.io/bugs.htm

El sitio web de Find Security Bugs describe las vulnerabilidades web más frecuentes y peligrosas en el código fuente con ejemplos.

© Universidad Internacional de La Rioja (UNIR)

OWASP ESAPI project

What is ESAPI? (s. f.). https://owasp.org/www-project-enterprise-security-api/

The OWASP Enterprise Security API (ESAPI) es una biblioteca de control de seguridad de aplicaciones web, gratuita y de código abierto que facilita a los programadores la escritura de aplicaciones de bajo riesgo. Las bibliotecas ESAPI están diseñadas para facilitar a los programadores la adaptación de la seguridad en las aplicaciones existentes. También sirven como una base sólida para nuevos desarrollos.

© Universidad Internacional de La Rioja (UNIR)

OWASP Java Encoder Project

OWASP Java Encoder. (s. f.). https://owasp.org/www-project-java-encoder/#tab=Use_the_Java_Encoder_Project

OWASP Java Encoder Project es una colección de codificadores contextuales de alto rendimiento y bajo costo que, cuando se utiliza correctamente, es una herramienta eficaz para prevenir la aplicación web.

© Universidad Internacional de La Rioja (UNIR)

1. ¿Cuál es el principal riesgo de una vulnerabilidad de tipo CWE-22 *(path traversal)*?

A. Inyección de código en el navegador.

B. Acceso no autorizado a archivos del sistema.

C. Exposición de contraseñas en texto plano.

D. Acceso concurrente a memoria compartida.

2. ¿Cuál es el principal impacto de una vulnerabilidad CWE-79 (XSS)?

A. Acceso directo a la base de datos.

B. Suplantación del sistema operativo.

C. Ejecución de *scripts* maliciosos en el navegador de la víctima.

D. Falsificación de certificados SSL.

3. ¿Cuál es una forma común de explotar CWE-89 (SQL Injection)?

A. Inyectar comandos Bash en una consulta.

B. Usar OR 1=1 en un campo de entrada.

C. Manipular cabeceras HTTP.

D. Forzar peticiones a API internas.

4. ¿Qué se necesita para explotar CWE-352 (CSRF)?

A. Que el usuario esté autenticado en una sesión válida.

B. Que la base de datos no tenga cifrado.

C. Que el navegador tenga *cookies* deshabilitadas.

D. Que el *firewall* esté inactivo.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Qué hace vulnerable a una aplicación frente a CWE-78 (Command injection)?

A. El uso de librerías JSON no seguras.

B. La ejecución de comandos del sistema con entradas no validadas.

C. El uso de *cookies* persistentes.

D. La presencia de código JavaScript externo.

6. ¿Qué implica la vulnerabilidad CWE-862?

A. Falta de cifrado en las contraseñas.

B. Exposición de rutas internas.

C. Acceso a funciones sin verificar permisos del usuario.

D. Uso de librerías desactualizadas.

7. ¿Qué problema puede causar CWE-434?

A. Sobrecarga del servidor.

B. Ejecución de archivos maliciosos subidos por el usuario.

C. Fuga de *tokens* de sesión.

D. Intercambio inseguro de claves.

8. ¿Cuál es una técnica para mitigar CWE-611?

A. Usar expresiones regulares.

B. Desactivar la carga de entidades externas en los analizadores XML.

C. Cambiar contraseñas periódicamente.

D. Usar solo *cookies* seguras.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Cuál es un riesgo típico de CWE-918 (SSRF)?

A. Capturar pulsaciones de teclas.

B. Obtener acceso a redes internas a través del servidor vulnerable.

C. Interceptar tráfico entre usuarios.

D. Inyección de SQL en segundo plano.

10. ¿Cuál es una consecuencia de CWE-798?

A. Los datos del usuario se cifran mal.

B. El sistema permite múltiples sesiones activas.

C. El atacante puede obtener acceso privilegiado fácilmente.

D. El servidor se reinicia inesperadamente.

© Universidad Internacional de La Rioja (UNIR)